

C Comments: The Complete Reference

A 16-Page Detailed Guide to Documentation & Code Clarity

Version 1.0 | Academic & Industry Standards

01. Introduction

Comments in C are non-executable statements used to explain, document, and describe the source code. They are completely ignored by the compiler, meaning they do not affect the program's logic, output, or performance.

Definition: A comment is a piece of text written inside a program to make the code easier to understand for humans.

02. Why Comments Exist?

Programming languages are written for humans first, machines second. Main purposes include readability, explaining complex logic, and helping others understand your thought process.

03. Importance in C Language

C is a low-level, close-to-hardware language, which means:

- Code can look cryptic to beginners.
- Logic (especially pointers) can be hard to follow.
- Memory management needs explicit explanation.

Hence, comments play a critical role in C. Code without comments is like a book without a table of contents or explanations.

04. Primary Types

C strictly supports two types of comments:

1. Single-line comments (`//`)

Modern C (C99 and later) supports the double-slash syntax for short annotations.

2. Multi-line comments (`/* */`)

The traditional C style used for block explanations and headers.

05. Single-Line Comments (//)

Introduced in the C99 standard, these are best for short inline explanations.

```
// This is a single-line comment  
int age = 20; // Storing the user's age
```

Characteristics:

Starts with // and ends automatically at the end of the physical line.

06. Multi-Line Comments (/* */) ---

Existing since the birth of C, these are essential for detailed notes.

```
/* This program calculates  
   the sum of two numbers  
   using user input */
```

Characteristics:

Requires an opening /* and a closing */. It can span dozens of lines.

07. Comparison Table

Feature	Single-line	Multi-line
Symbol	//	/* */
Lines	One	Multiple
Use case	Short notes	Long explanations
Standard	C99 onwards	Early C (K&R)

08. Placement in Code

Comments are highly flexible and can be placed:

- **Before statements:** To explain what's coming.
- **After statements:** (Inline) To explain a specific variable.
- **Headers:** At the very top of the .c file for metadata.

```
#include <stdio.h> // Header file  
  
int main() { ... }
```


09. Header & Documentation

In industry and academia, header blocks are mandatory.

```
/*  
 * Program: Matrix Multiplier  
 * Author: Avni Sharma  
 * Date: Feb 2026  
 */
```

This serves as the "Identity Card" of your source code.

10. Function & Block Levels

Function Level: Describes parameters and return values.

Block Level: Describes the logic of an algorithm (e.g., Sorting or Searching steps).

Always comment the "Why" of an algorithm, not just the "How".

11. Using Comments for Debugging

Comments are a programmer's best friend during troubleshooting. You can "Comment Out" code to isolate errors.

```
// printf("Checking variable x...");  
/*  
old_logic_function();  
*/
```

This prevents the code from running without having to delete it permanently.

12. Nested Comments (IMPORTANT)

C does **NOT** support nested multi-line comments. This is a common exam question and a frequent source of "cryptic" compiler errors.

```
/* Outer comment  
    /* Inner comment */  
*/ <-- This will cause a Syntax Error!
```

13. Best Practices

- ✓ **Meaningful:** Explain the purpose of a block.
- ✓ **Updated:** If you change the code, change the comment.
- ✗ **Obvious:** Avoid `int x = 5; // set x to 5.`
- ✗ **Cluttered:** Don't let comments overwhelm the actual code.

14. Industry Standards

Advanced tools like **Doxygen** scan your C comments to generate professional PDF/HTML manuals automatically.

```
/**  
 * @brief Calculates Sum  
 * @param a First int  
 */
```

15. Global Comparison

How C style compares to others:

- **C / C++ / Java / JS:** Use `//` and `/* */`
- **Python:** Uses `#`
- **HTML:** Uses `<!-- -->`

C set the standard for the most popular bracket-style languages in history.

16. Summary & Exam Prep

Q: Are comments executable? **No.**

Q: Can you nest /* */? **No.**

Q: Which standard added //? **C99.**

One-Line Definition: Comments are non-executable explanations that make code readable and maintainable.

END OF GUIDE